

Optimizers

- Optimizers in deep learning play a crucial role in updating the weights and biases of the model **to minimize the loss function**.
- **Gradient Descent (GD)**
- **Batch Gradient Descent**
- **Stochastic Gradient Descent (SGD)**
- **Mini-Batch Gradient Descent**
- **Adagrad**
- **RMSProp**
- **Adam (Adaptive Moment Estimation)**

What is a Gradient?

- A **gradient** is a measure of **how much the output of a function changes when you change the input**.
- In deep learning, a **gradient** means:
- How much **the loss function** will change if we slightly change a parameter (like a weight or bias).
- It shows the **direction and rate** of change.
- **For Flat Surface:** Gradient = 0 → No change needed.
- The **gradient is 0**, which means the model **stops learning**.
- This is called the **vanishing gradient problem in Deep Learning**.
- **For Sloped Surface**
- **Positive gradient** → Decrease the weight.
Negative gradient → Increase the weight.

Gradient Descent (GD)

- Gradient descent is a simple optimization algorithm that updates the model's parameters **to minimize the loss function**.
- **Steps in Gradient Descent:**
- **Calculate Gradient:** Compute the slope (gradient) of the loss function with respect to the **initial parameters(weights and bias)**.
- **Update Parameters:** Adjust the parameters in the opposite direction of the gradient to move closer to the minimum.

$$\text{New parameter} = \text{Current parameter} - \text{Learning rate} \times \text{Gradient}$$

- **Repeat:** Continue this process until the loss function becomes very small(minimum).

Gradient Descent (GD)

$$\theta = \theta - \eta \cdot \nabla L(\theta)$$

θ : Parameter to be updated (e.g., weight or bias).

η : Learning rate (step size).

$\nabla L(\theta)$: Gradient of the loss function with respect to θ .

the gradient descent update equation for weights is written as:

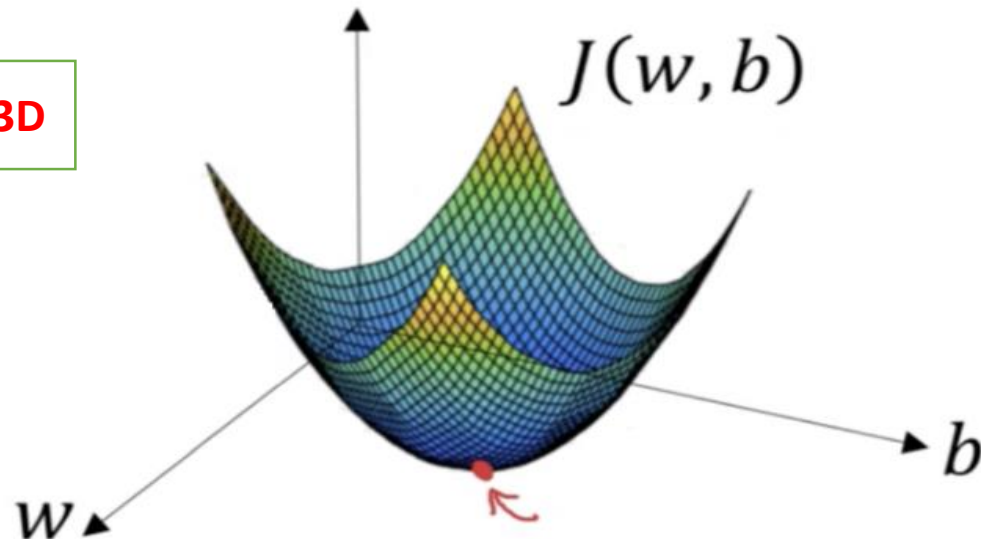
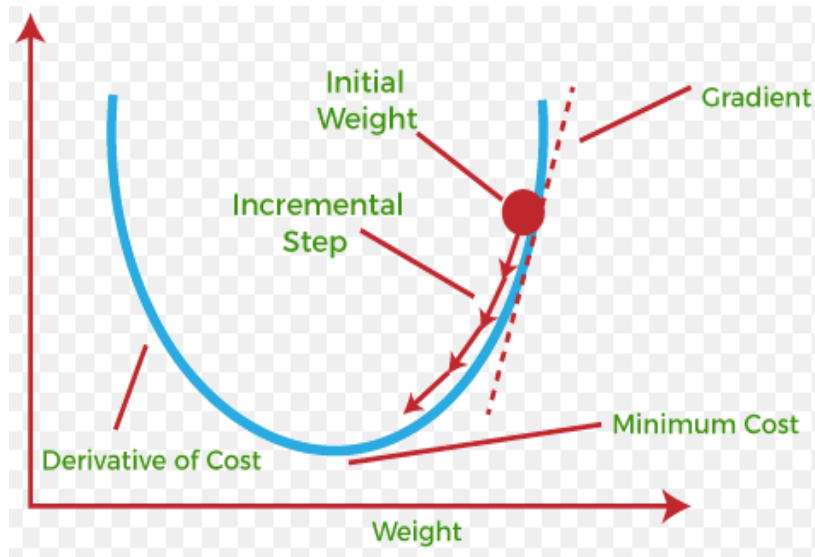
$$w = w - \eta \cdot \frac{\partial L}{\partial w}$$

Where:

- w : Weight parameter to be updated.
- η : Learning rate (step size).
- $\frac{\partial L}{\partial w}$: Gradient of the loss function (L) with respect to the weight (w).

For bias update

$$b = b - \eta \cdot \frac{\partial L}{\partial b}$$



Example

- Loss Function:

$$L(w) = (w - 3)^2$$

Steps:

1. **Initialize Parameter:** Start with $w = 0$.
2. **Compute Gradient:** Derivative of $L(w)$ with respect to w is:

$$\frac{\partial L}{\partial w} = 2(w - 3)$$

3. **Update Parameter:** Using the update rule:

$$w_{\text{new}} = w_{\text{current}} - \eta \cdot \frac{\partial L}{\partial w}$$

Let $\eta = 0.1$.

- Iteration 1:

$$\frac{\partial L}{\partial w} = 2(0 - 3) = -6$$

$$w_{\text{new}} = 0 - 0.1 \cdot (-6) = 0.6$$

- Iteration 2:

$$\frac{\partial L}{\partial w} = 2(0.6 - 3) = -4.8$$

$$w_{\text{new}} = 0.6 - 0.1 \cdot (-4.8) = 1.08$$

- Iteration 3:

$$\frac{\partial L}{\partial w} = 2(1.08 - 3) = -3.84$$

$$w_{\text{new}} = 1.08 - 0.1 \cdot (-3.84) = 1.464$$

The process continues until the gradient becomes close to 0

Example 2

- Consider a simple linear model: $y = w \cdot x + b$

Given Data:

- Input (x): 2
- True output (y_{true}): 4
- Initial weight (w): 1
- Initial bias (b): 0
- Learning rate (η): 0.1

The **loss function** is the Mean Squared Error (MSE):

$$L = \frac{1}{2}(y - y_{\text{true}})^2$$

where:

- w : Weight
- b : Bias
- x : Input
- y : Predicted output

Step 1: Forward Pass

Compute the predicted output y :

$$y = w \cdot x + b = 1 \cdot 2 + 0 = 2$$

Compute the loss:

$$L = \frac{1}{2}(y - y_{\text{true}})^2 = \frac{1}{2}(2 - 4)^2 = \frac{1}{2} \cdot 4 = 2$$

Step 2: Backward Pass

1. Compute the gradient of the loss with respect to w ($\frac{\partial L}{\partial w}$):

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial w}$$

$$\frac{\partial L}{\partial y} = (y - y_{\text{true}}) = (2 - 4) = -2$$

$$\frac{\partial y}{\partial w} = x = 2$$

$$\frac{\partial L}{\partial w} = -2 \cdot 2 = -4$$

2. Compute the gradient of the loss with respect to b ($\frac{\partial L}{\partial b}$):

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial b}$$

$$\frac{\partial L}{\partial b} = -2 \cdot 1 = -2$$

The derivative of b with respect to itself is 1

Step 3: Update Weights and Bias

1. Update w :

$$w = w - \eta \cdot \frac{\partial L}{\partial w}$$

$$w = 1 - 0.1 \cdot (-4) = 1 + 0.4 = 1.4$$

2. Update b :

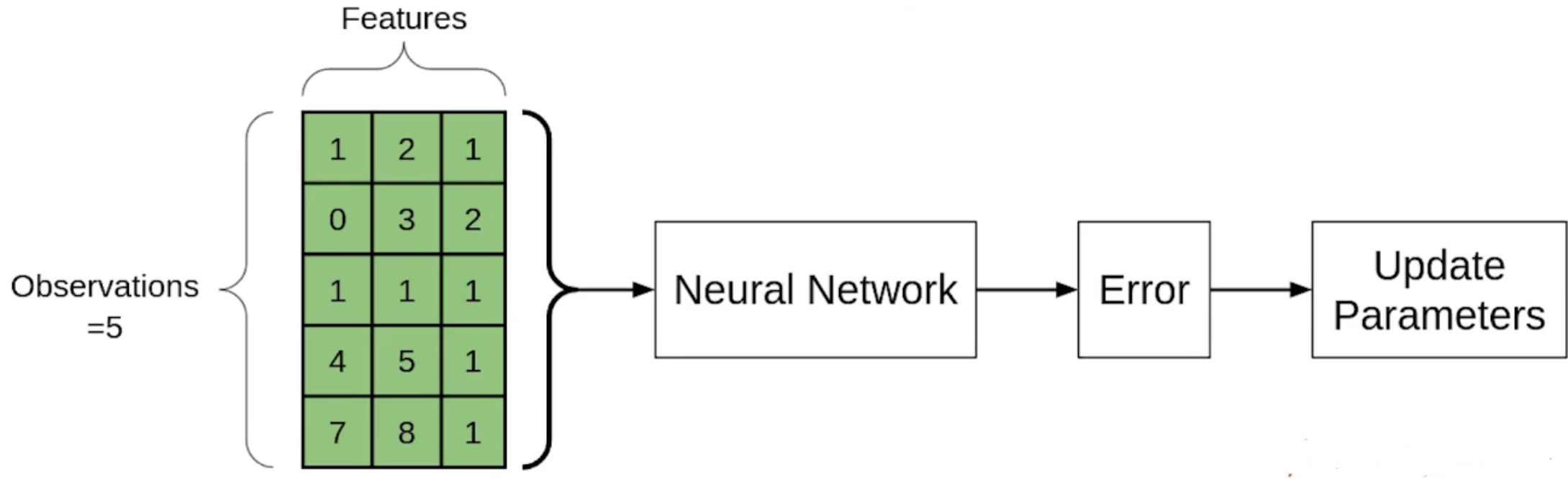
$$b = b - \eta \cdot \frac{\partial L}{\partial b}$$

$$b = 0 - 0.1 \cdot (-2) = 0 + 0.2 = 0.2$$

Types of Gradient Descent

- There are three main variations of Gradient Descent:
- **Batch Gradient Descent**
- Uses the entire dataset to compute the gradient.
- **Stochastic Gradient Descent (SGD)**
- Computes the gradient using only one data point at each step.
- **Mini-batch Gradient Descent**
- Uses a small batch of data points to compute the gradient at each step.

Batch Gradient Descent

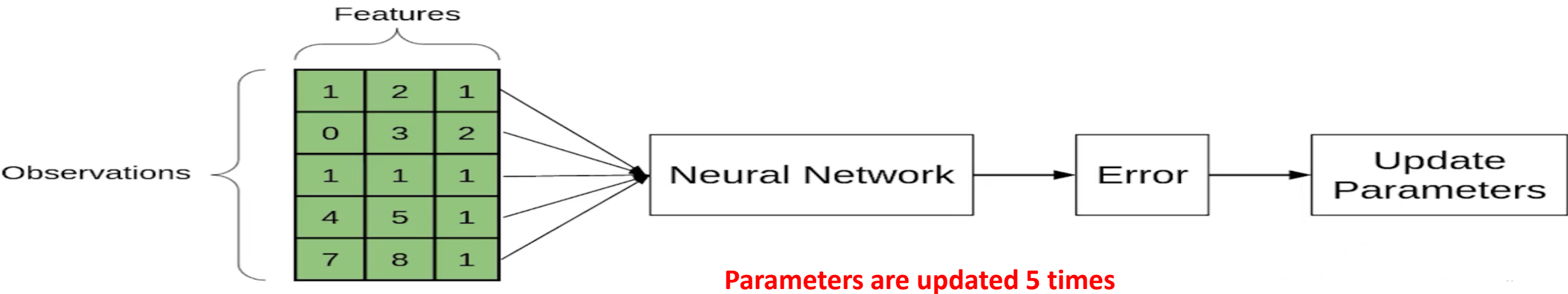
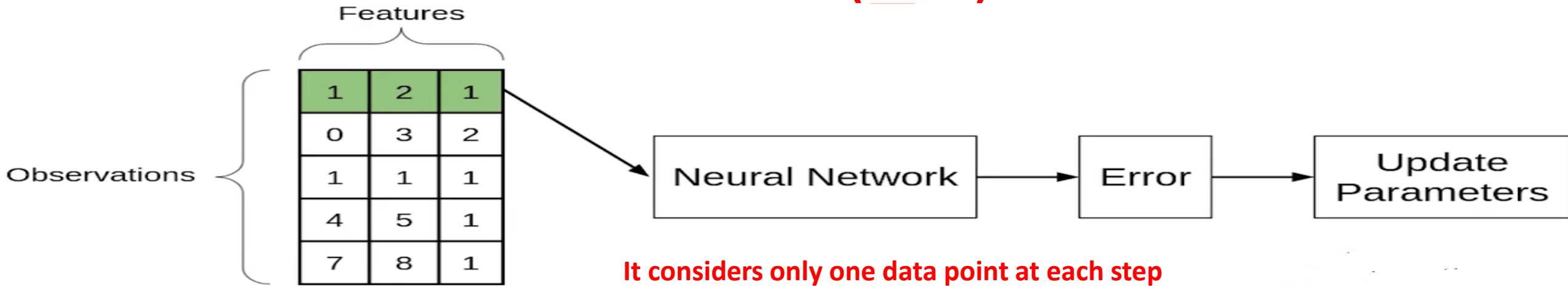


Parameters are updated only once.

Advantage: Accurate gradient.

Disadvantage: Computationally expensive for large datasets.

Stochastic Gradient Descent (SGD)

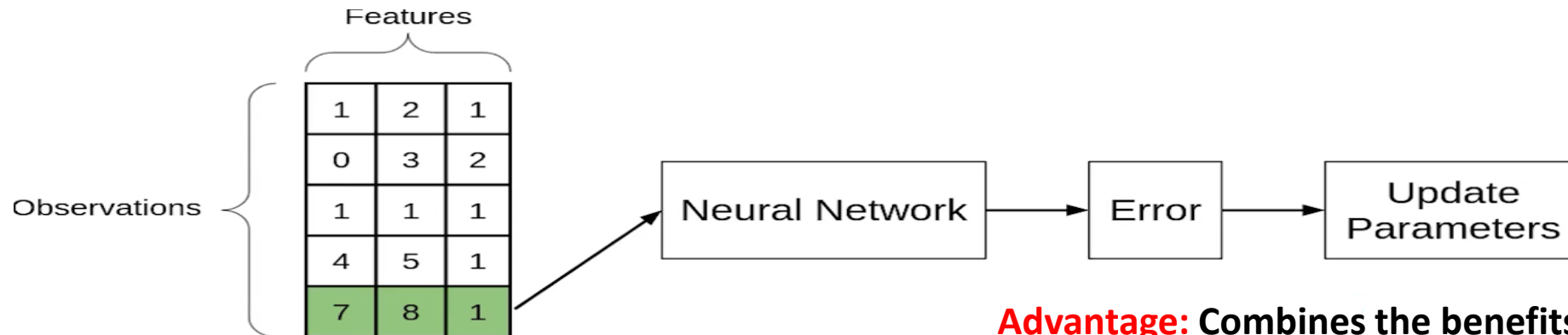
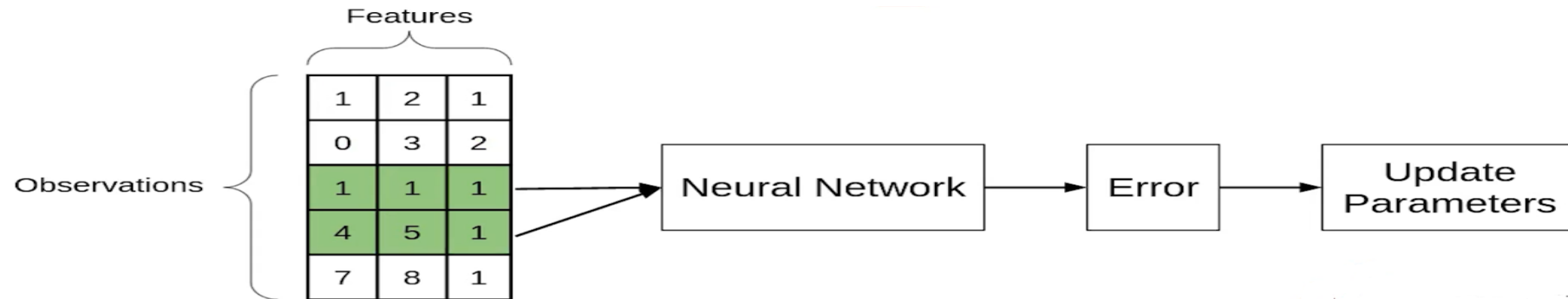
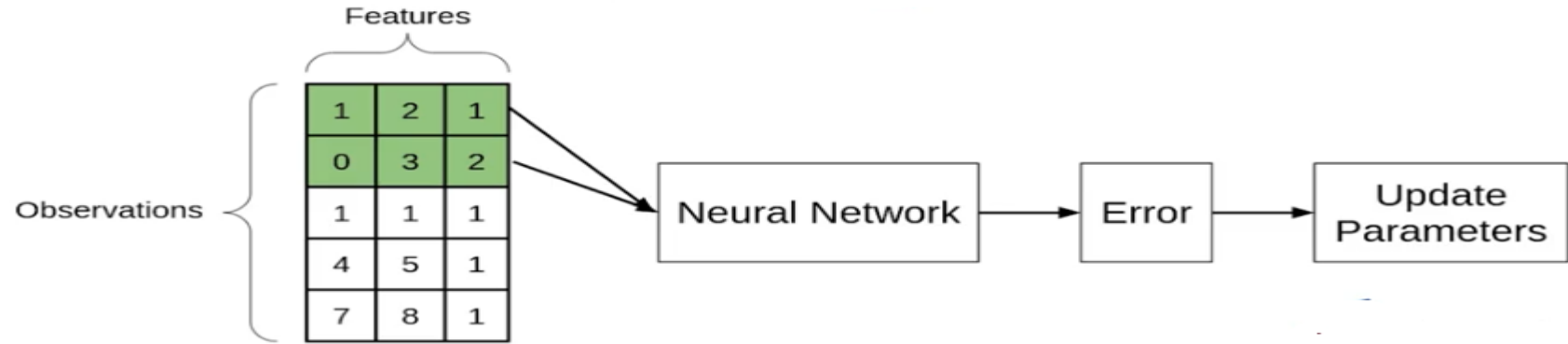


Advantage: Faster and requires less memory.

Disadvantage: Can be noisy and may not converge smoothly.

Mini-batch Gradient Descent

Batch Size = 2

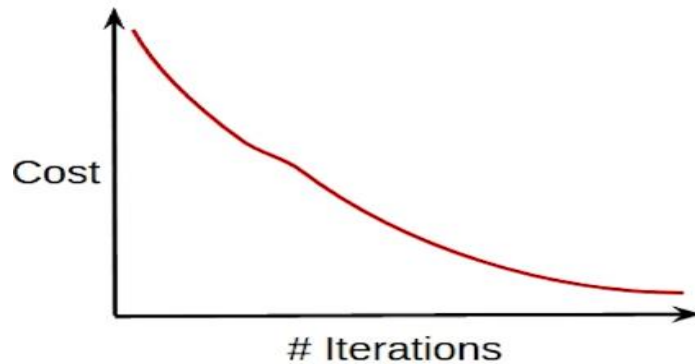


Advantage: Combines the benefits of Batch and SGD, commonly used in practice.

Comparison: Cost Function

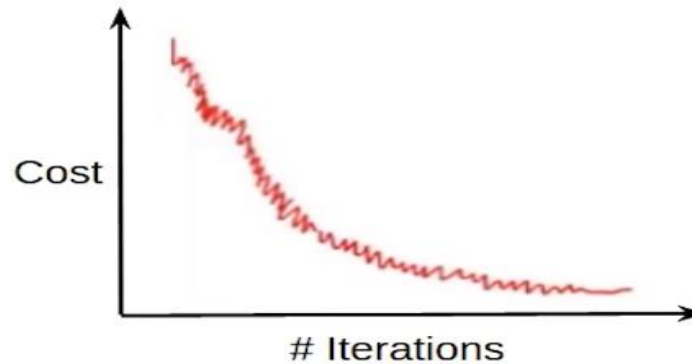
Batch Gradient Descent

- Entire dataset for updation
- Cost function reduces smoothly



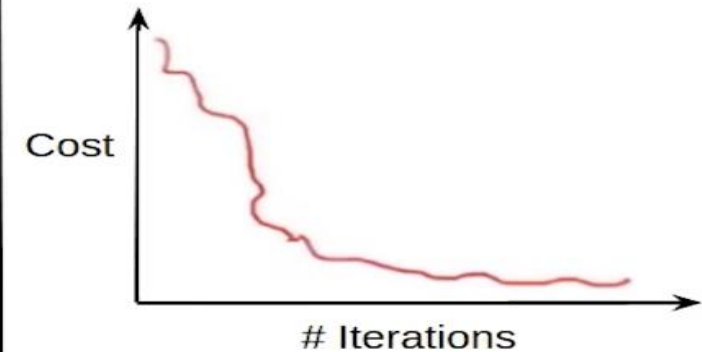
Stochastic Gradient Descent (SGD)

- Single observation for updation
- Lot of variations in cost function



Mini-Batch Gradient Descent

- Subset of data for updation
- Smoother cost function as compared to SGD



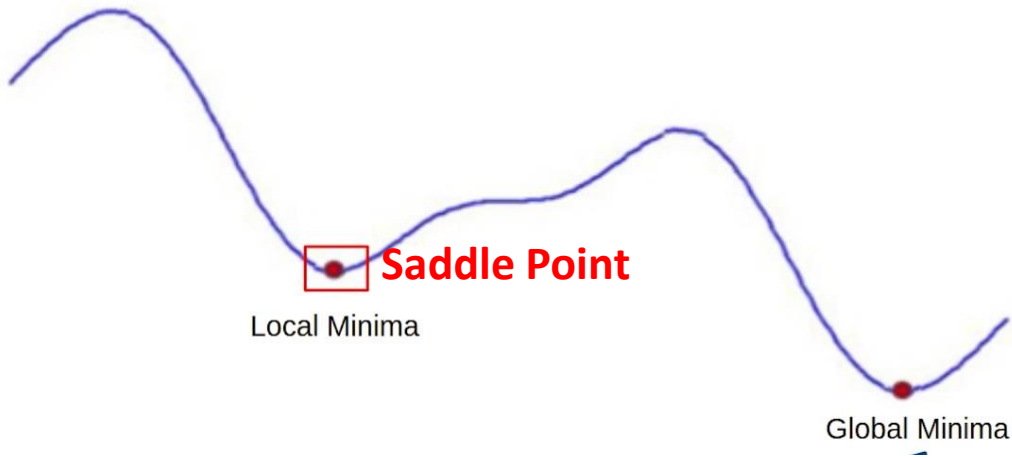
- Computation cost is very high

- Computation time is more

- Computation time is lesser than SGD
- Computation cost is lesser than Batch Gradient Descent

Problems/Challenges with Gradient Descent

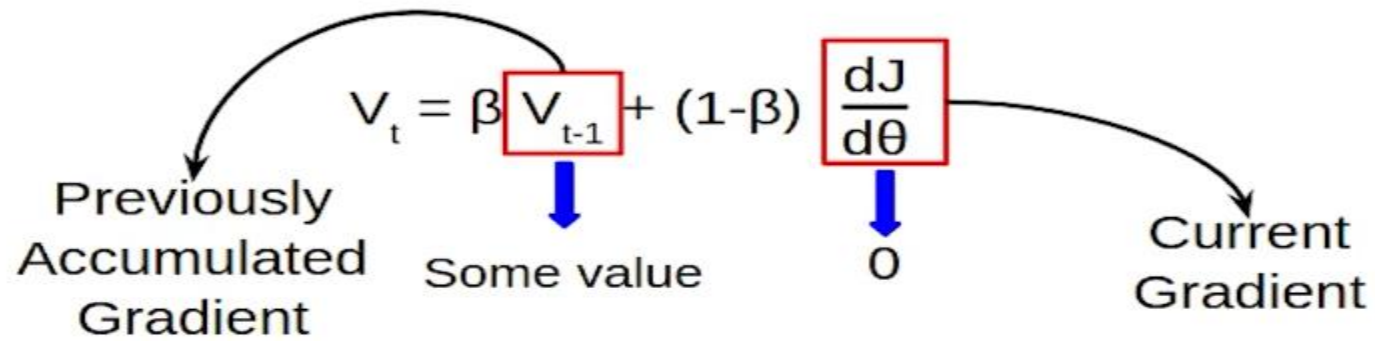
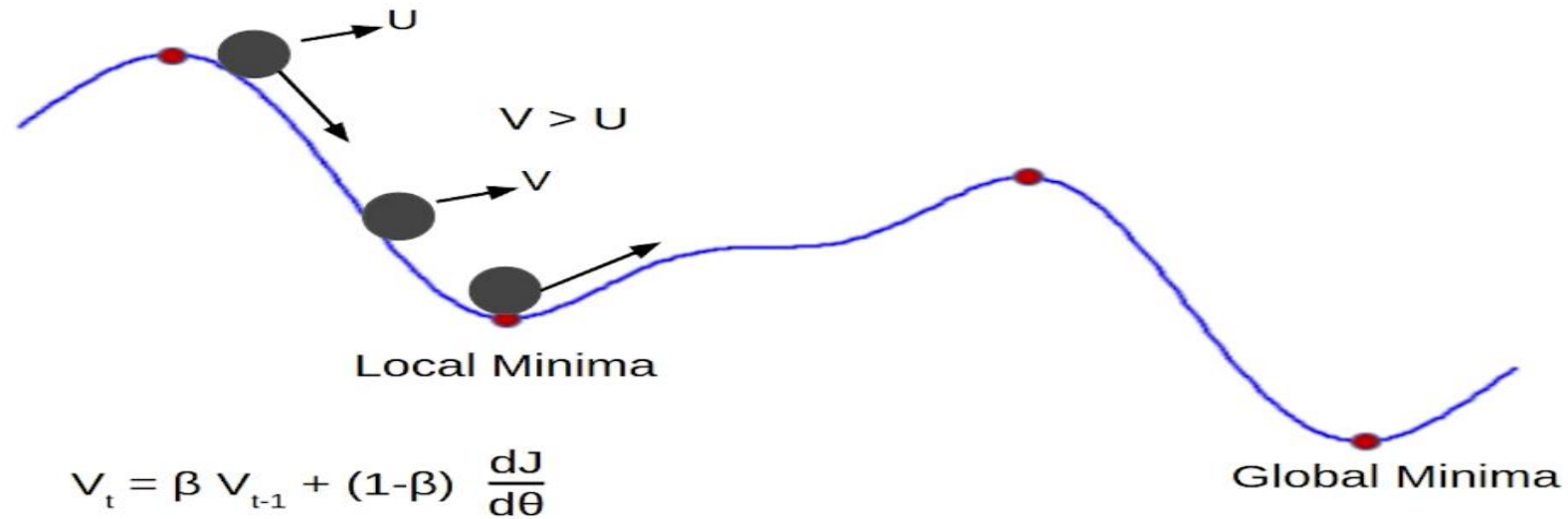
1. Getting stuck at local minima



$$\theta_i = \theta_i - \alpha * \boxed{dJ / d\theta_i}$$

Solution for Stuck at local minima(Stochastic Gradient Descent with Momentum)

Weighted Gradient



$$V_t = \beta V_{t-1} + (1-\beta) \frac{dJ}{d\theta}$$

$\beta = 0.9$

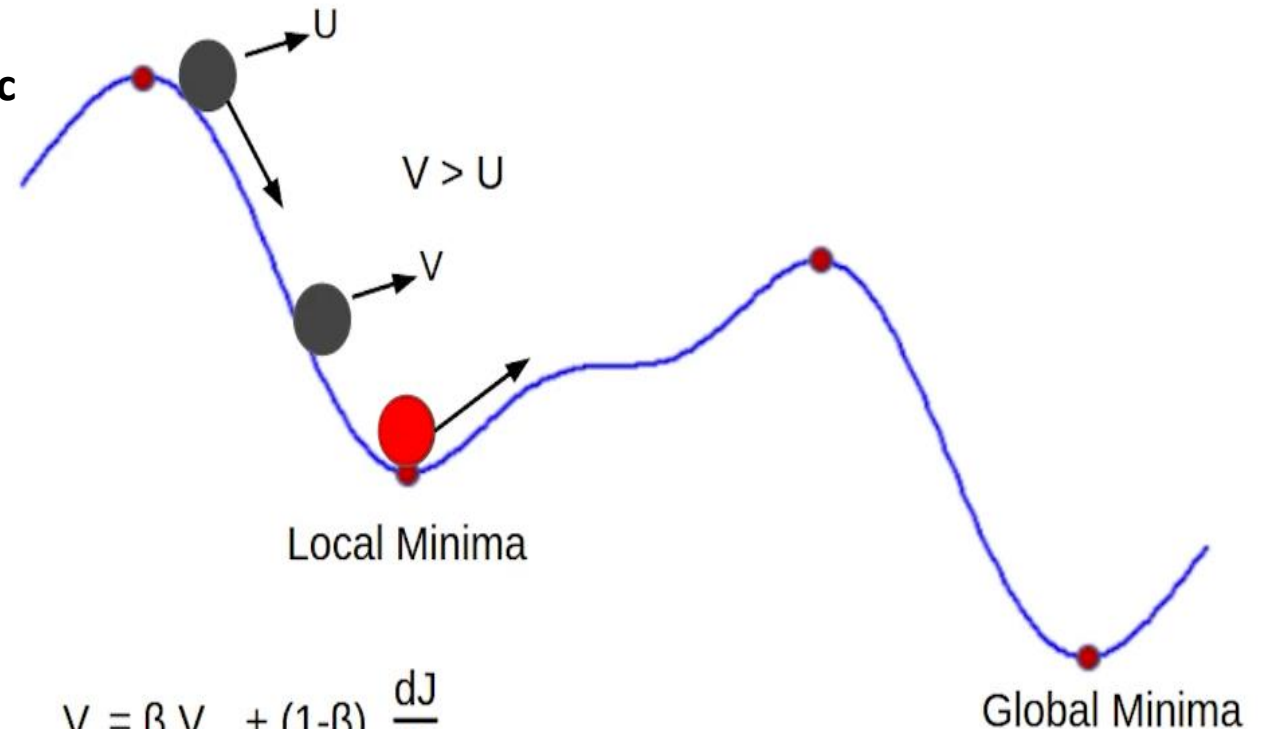
- 10% weight to current gradient
- 90% weight to previous gradient

Stochastic Gradient Descent with Momentum

SGD with Momentum is an enhancement of **Stochastic Gradient Descent (SGD)** that helps the optimization process converge faster by reducing oscillations and improving stability during training.

Advantages of SGD with Momentum

- Reduces Oscillations
- Accelerates Convergence
- Overcomes Local Minima



$$V_t = \beta V_{t-1} + (1-\beta) \frac{dJ}{d\theta}$$

$$\theta_i = \theta_i - \alpha * V_t$$

Problems with Gradient Descent

1. Getting stuck at local minima
2. Same Learning rate throughout the training process

Adagrad Optimizer

- Adagrad stands for **Adaptive Gradient**
- It adjusts the learning rate for each parameter based on the history of the gradients.
- Adagrad dynamically adjusts learning rates over time.
- **accumulated gradient** is denoted as G_t

$$G_t = G_{t-1} + g_t^2$$

- Here:
- G_{t-1} is the sum of squared gradients up to the previous time step.
- g_t^2 is the square of the current gradient for the parameter.

- Update the Parameters by using

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot g_t$$

η is the initial learning rate

G_t is the accumulated squared gradients up to the time step t .

ϵ is a small value to avoid division by zero.

g_t is the gradient at the current time step.

It is good choice for applications like **natural language processing, computer vision** where sparse gradients are common.

Drawback:

As **G_t** grows larger with each iteration, the denominator becomes very large, causing the effective learning rate to **approach zero**.

This leads to:

- **Slow convergence** in later stages of training.
- The optimizer getting **stuck in suboptimal solutions**.
- **Ineffective learning for deep neural networks and large datasets**.

AdaDelta (Adaptive Delta) Optimization Algorithm

- The **AdaDelta** (Adaptive Delta) algorithm is an extension of the **AdaGrad** optimizer.
- It aims to address some of its limitations of **AdaGrad** optimizer.
- **AdaDelta** modifies Adagrad by using a **decaying average of squared gradients** to ensure that the learning rate doesn't vanish.
- It well-suited for optimizing deep learning models where the learning rate needs to remain effective over a long period.
- Adadelta restricts the window of accumulated past gradients to a fixed size **w**.
- AdaDelta **automatically adjusts the learning rate** based on the gradients.

RMSProp(Root Mean Square Propagation)

Problem: Same learning rate for all parameters

$$\eta_t = \sum_{i=1}^{t-1} \left[\frac{dJ}{d\theta} \right]^2$$

$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{\eta_t + \epsilon}} \frac{dJ}{d\theta}$$

Where:

- α is the learning rate.
- ϵ is a small constant added for numerical stability (e.g., 10^{-8}).
- η_t is the moving average of the squared gradients.



- **Real-World Example: Filling a Swimming Pool**
- Imagine filling a **swimming pool** with **multiple water pipes**:
- **Assume Each pipe represents a model parameter** (like weights in neural network).
- **Water flow represents the gradients.**
- The goal is to **fill the pool (reach the optimal parameters)** without **overflow or underflow**.
- If a pipe has been flowing a lot of water recently (large gradient), RMSProp reduces the water flow for that pipe.
- If a pipe hasn't been flowing much water recently (small gradient), RMSProp increases the water flow.
- The optimizer keeps track of **recent water flows** for each pipe and **adjusts the flow rate dynamically** to maintain a steady fill.

Key features of RMSProp

- **RMSProp adapts the learning rate for each parameter independently.**
- **Parameters with large gradients get a smaller learning rate.**
- **Parameters with small gradients get a larger learning rate.**
- It helps the model:
 - **Converge faster.**
 - **Avoid exploding gradients.**
 - **Handle sparse gradients better.**

$$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\eta}} \frac{dJ}{d\theta}$$

Tends to Zero

$$\theta_i \approx \theta_{i-1}$$

Leads to Slower Convergence

$$\mu_t = \beta \mu_{t-1} + (1-\beta) \left[\frac{dJ}{d\theta} \right]^2$$

$$\theta_i = \theta_i - \frac{\alpha}{\sqrt{\mu_t + \epsilon}} \frac{dJ}{d\theta}$$

$$\mu_t = \beta \mu_{t-1} + (1-\beta) \left[\frac{dJ}{d\theta} \right]^2$$

High

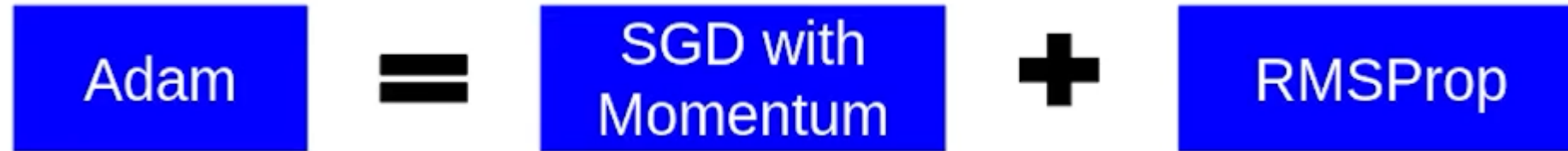
Low

$$\theta_i = \theta_i - \frac{\alpha}{\sqrt{\mu_t + \epsilon}} \frac{dJ}{d\theta}$$

Low

High

Adam Optimizer (Adaptive Moment Estimation)



Adam is one of the most widely used optimization algorithms in **deep learning**

It combines the best ideas from **SGD with Momentum** and **RMSProp** to achieve **fast convergence** and **better stability**.

Adam optimizer acts like a **smart GPS** that helps to reach the destination (optimal parameters) quickly and efficiently. Adjusting the **speed** (learning rate) based on recent progress.

Adam

=

SGD with
Momentum

+

RMSProp

$$v_t = \beta_1 v_{t-1} + (1-\beta_1) \frac{dJ}{d\theta}$$

$$\beta_1 = 0.9$$

90% weight to the past gradient and 10% to the current gradient.

$$\mu_t = \beta_2 \mu_{t-1} + (1-\beta_2) \left[\frac{dJ}{d\theta} \right]^2$$

$$\beta_2 = 0.999$$

99.9% weight to the past squared gradients.

$$\theta_i = \theta_{i-1} - \frac{\alpha}{\sqrt{\mu_t} + \epsilon} * v_t$$

Why is Adam So Popular?

- **It's fast** — Adam converges faster than other optimizers like SGD.
- **It's reliable** — It handles noisy gradients, sparse data.
- **It's adaptive** — No need to manually tune the learning rate during training.
- **Widely Used in Practice:**
 - Works well for a variety of deep learning tasks like NLP, Computer Vision, and Reinforcement Learning.

Backpropagation

- It allows the network to **learn from its mistakes** by adjusting its **weights and biases** to reduce the error between the predicted output and the actual output.

- **Why Backpropagation?**

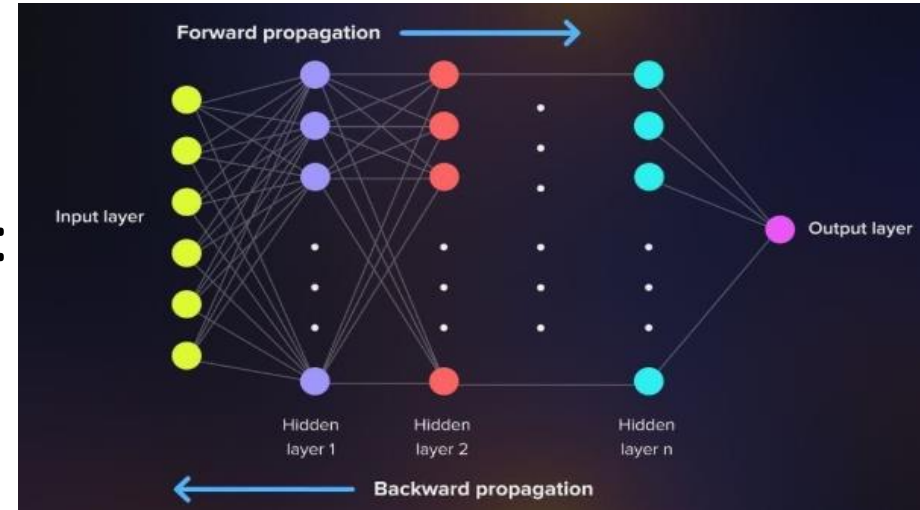
- Training a neural network involves **two** steps:

- **Forward Pass:**

The input is passed through the network to generate the output.

- **Backward Pass (Backpropagation):**

The network compares the output with the actual value, calculates the **error**, and adjusts the **weights** to reduce this error.



Key Idea Behind Backpropagation:

- The key idea is to use **the chain rule from calculus** to calculate how much each weight in the network contributed to the final error. Then, we adjust each weight to **minimize that error**.